

inlabru: Bayesian spatial and spatio-temporal modelling in R

Finn Lindgren

finn.lindgren@ed.ac.uk



10 March 2026

inlabru software interface concepts

- Model components are declared similarly to R-INLA:

```
# INLA:
~ covar + f(name, model = ...)
# inlabru
~ covar + name(input, model = ...)
~ covar # is translated into...
~ covar(covar, model = "linear")
~ name(1) # Used for intercept-like components
```

- In R-INLA, $\eta = \mathbf{A}\mathbf{u} = \mathbf{A}_0 \sum_{k=1}^K \mathbf{A}_k \mathbf{u}_k$, where the rows of $\mathbf{A}_k$ only extract individual elements from each $\mathbf{u}_k$, and the overall $\mathbf{A}_0$ is user defined (via `inla.stack()`).
- In inlabru, $\eta = h(\mathbf{u}_1, \dots, \mathbf{u}_K, \mathbf{A}_1 \mathbf{u}_1, \dots, \mathbf{A}_K \mathbf{u}_K)$, where $h(\cdot)$ is a general R expression of named latent components $\mathbf{u}_k$ and intermediate "effects" $\mathbf{A}_k \mathbf{u}_k$
- $\mathbf{A}_k$ by default acts either as in R-INLA, or is determined by a *mapper* method. Predefined default mappers include e.g. spatial evaluation of SPDE/GRMF models that map between coordinates and meshes, and mappers that combine other mappers (used to combine main/group/replicate for all components)

Input mappers

- Each named component has main/group/replicate *inputs*, that are given to the mappers to evaluate A_k . For a given latent *state*, the resulting *effect* values are made available to the predictor expression.

```
bru_get_mapper() # Obtain default mapper for a model object
bm_index(n) # Basic index mapping
bm_linear() # Basic linear mapping
bm_matrix(labels) # Basic linear multivariable mapping
bm_factor(values, factor_mapping) # Factor variable mapping
bm_multi(mappers) # Kronecker product components
bm_collect(mappers, hidden) # For concatenated components, like bym

bm_const() # Constants
bm_scale() # Fixed scaling
bm_marginal() # Marginal distribution transformation
bm_aggregate()/logsumexp() # Weighted block-wise sum or log-sum-exp
bm_pipe() # Composition of mappers

bm_fmasher(mesh) # 1D/2D/3D and spherical mesh mappings
bru_mapper.fm_mesh_1d(mesh) # Indexed 1D mappings
```

■ Model component definition examples:

```
comp <- ~ -1 + field(cbind(easting, northing), model = spde) + param(1) # Raw data
comp <- ~ -1 + field(geometry, model = spde) + param(1) # sf data
```

■ Predictor formula examples, including naming of the response variable:

```
form1 <- my_counts ~ param + field
form2 <- response ~ exp(param) + exp(field)
```

■ Main method call structure:

```
bru(components = comp,
    bru_obs(formula = form1, family = "poisson", data = data1),
    bru_obs(formula = form2, family = "normal", data = data2))
```

■ Simplified notations for common special cases;

```
formula = response ~ .
```

gives the full additive model of all the available components, like

```
formula = response ~ field + param
```

Plain INLA code for a separable space-time model

```

matern <- inla.spde2.pcmatern(mesh, ...)

field_A <- inla.spde.make.A(mesh,
                             data,
                             group = data$year - min(data$year) + 1,
                             n.group = 10)
field_index <- inla.spde.make.index("field", n.spde = matern$n.spde, n.group = 10)
stk <- inla.stack(data = list(response = data$response),
                 A = list(field_A, 1),
                 effects = list(field_index, list(covar = data$covar)))

formula <- response ~ 1 + covar +
  f(field, model = matern, group = field_group, control.group = ...)

fit <- inla(formula = formula,
            data = inla.stack.data(stk, matern = matern),
            family = "normal",
            control.predictor = list(A = inla.stack.A(stk)))

```

inlabru code for a separable space-time model

```
matern <- inla.spde2.pcmatern(mesh, ...)  
  
year_mapper <- bru_mapper(fm_mesh_1d(sort(unique(data$year))), indexed = TRUE)  
  
comp <- ~  
  Intercept(1) +  
  covar +  
  field(geometry,  
    model = matern,  
    group = year, group_mapper = year_mapper, control.group = ...)  
  
fit <- bru(components = comp,  
  bru_obs(response ~ .,  
    data = data,  
    family = "normal"))
```

inlabru code for a non-separable space-time model

```
model <- INLAspacetime::stModel.define(smesh, tmesh, ...)
# The INLAspacetime package defines a bru_get_mapper() method for the stModel class,
# based on the smesh and tmesh meshes, so no manual mapper definition is needed

comp <- ~
  Intercept(1) +
  covar +
  field(list(space = geometry, time = year),
        model = model)

fit <- bru(components = comp,
           bru_obs(response ~ .,
                   data = data,
                   family = "normal"))
```

Plain INLA code for a point process model, with prediction

```

field_index <- inla.spde.make.index("field", n.spde = matern$n.spde, n.group = 10)
integ <- fm_int(list(geometry = fm_subdivide(mesh, 1)), boundary)
A.points <- fm_basis(mesh, points) # Or inla.spde.make.A(mesh, points)
A.integ <- fm_basis(mesh, integ) # Or inla.spde.make.A(mesh, integ)
A.pred_points <- fm_basis(mesh, pred_points) # Or inla.spde.make.A(mesh, pred_points)
covar.points <- eval_spatial(covar_raster, points)
covar.integ <- eval_spatial(covar_raster, integ)
covar.pred <- eval_spatial(covar_raster, pred_points)
stack.points <- inla.stack(list(Y = rep(1, nrow(points)), E = rep(0, nrow(points))),
                           A = list(A.points, 1),
                           effects = list(field = field_index,
                                           list(Intercept = 1, covar = covar.points)))
stack.integ <- inla.stack(list(Y = rep(0, nrow(integ)), E = integ$weight),
                          A = list(A.integ, 1),
                          effects = list(field = field_index,
                                           list(Intercept = 1, covar = covar.integ)))

```

(continued...)

Plain INLA code for a point process model, with prediction (continued)

```
stack.pred <- inla.stack(list(Y = rep(NA, nrow(pred_points))),  
                        A = list(A.pred_points, 1),  
                        effects = list(field = field_index,  
                                      list(Intercept = 1, covar = covar.pred)),  
                        tag = "pred")  
stk <- inla.stack(stack.points, stack.integ, stack.pred)  
fit <- inla(Y ~ Intercept + field + covar),  
          data = inla.stack.data(stk, matern = matern),  
          family = "poisson",  
          control.predictor = list(A = inla.stack.A(stk), link = 1))  
pred.idx <- inla.stack.index(stk, tag = "pred")$data  
prediction <- fit$summary.fitted.values[pred.idx, ]
```

inlabru code for a point process model, with prediction

```
fit <- bru(geometry ~ Intercept(1) +  
           field(geometry, model = matern) +  
           covar(covar_raster),  
           data = points,  
           family = "cp",  
           domain = list(geometry = fm_subdivide(mesh, 1)),  
           samplers = boundary)  
prediction <- predict(fit, pred_points, ~ exp(Intercept + field + covar))
```